

Motivations

I want to work with NeuroKnowIA as a Front-end developer because I take it as a challenge. Being a Front-end developer requires detail and creativity for me. To work as a front-end developer on a B2B SaaS project, it is necessary to have an understanding of client needs and expectations. Secondly, NeuroKnowAI is a startup, and so it's on its way to evolve. I am motivated to take this position because I want to have an impact on its development and the company's project and vision through my competence. And finally, I aim to work internationally using the English language. My future professional goals align with the opportunity to use English during this internship experience.

My design for building a B2B application is to understand the user's needs and the application service first. After that, I can decide how to display and process the data in the application. The design of the application is to make it simple to use and easy to understand.

To balance the functionality and the aesthetics of the application, the UI part has to use at least three colors and have a frame and behavior that users understand in daily life. The application's basic functionality should be visualized in the application in the simplest and easiest manner possible. Therefore, the UX is essential for the application's success and the user's satisfaction; too much information confuses the user, and the visual hard experience to understand can be irritating.

UI/UX Theory questions

1-Privacy-first design: For an application where privacy is a default feature, I would consider how to access the data to secure the user's information and avoid any security issues on the internet. Furthermore, the application should include activity tracking in the data to manage its accessibility and security.

3 Examples for NeuroKnowIA case:

1- Privacy level options: The data accessing has many options; with these options, the user can access the data in the privacy that he/she is allowed to, depending on the user accessibility and account.

2- Private Routing: All the pages in the application have their privacy level; in that way, the data is protected; the user can view only the data that he/she can access.

3-Data Masking: The privacy level option and this feature work together to mask data according to the data options and the user's account. Either the sensitive information is detected and replaced with a symbol (e.g., '***') or the sensitive data is removed from the screen.

2- Progressive disclosure:

When an application has a user that has a different use, the best solution to avoid the overwhelming information is to set the default to the one who has less information. In this case, all the users have access to the basic data in the application when they open it; with this method, the one who uses the advanced information has an option to visualize the data by activating the advanced option. Furthermore, the application design should have in mind a way to separate the user's needs and protect the data; it may be useless and overwhelming for one of the users. The other method to separate the data depending on the users' wants is using a private route that depends on the user's account so they can have the information's requirements.

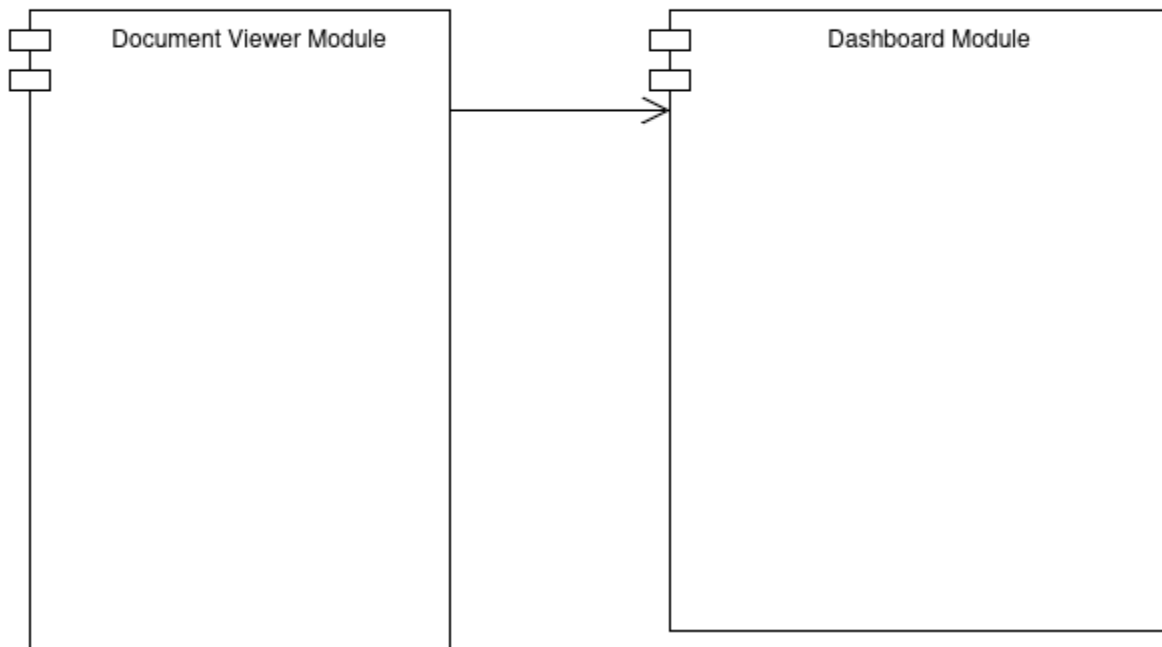
3- Error recovery:

When a user uploads 100 documents, 15 failed due network issues, for this issue, the design should report the upload result with the successful and failed task showing the file name and the reason for the failing. And after that, asking the user to try to upload the other files again without re-uploading the file. During the upload, it's necessary to display the file upload progression so that the user is updated of the uploading process.

Problem 4: Frontend Architecture Documentation

Architecture Diagram

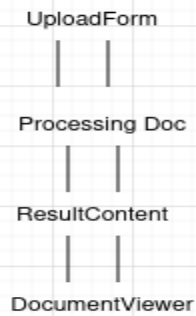
NeuroKnowIA Document Viewer



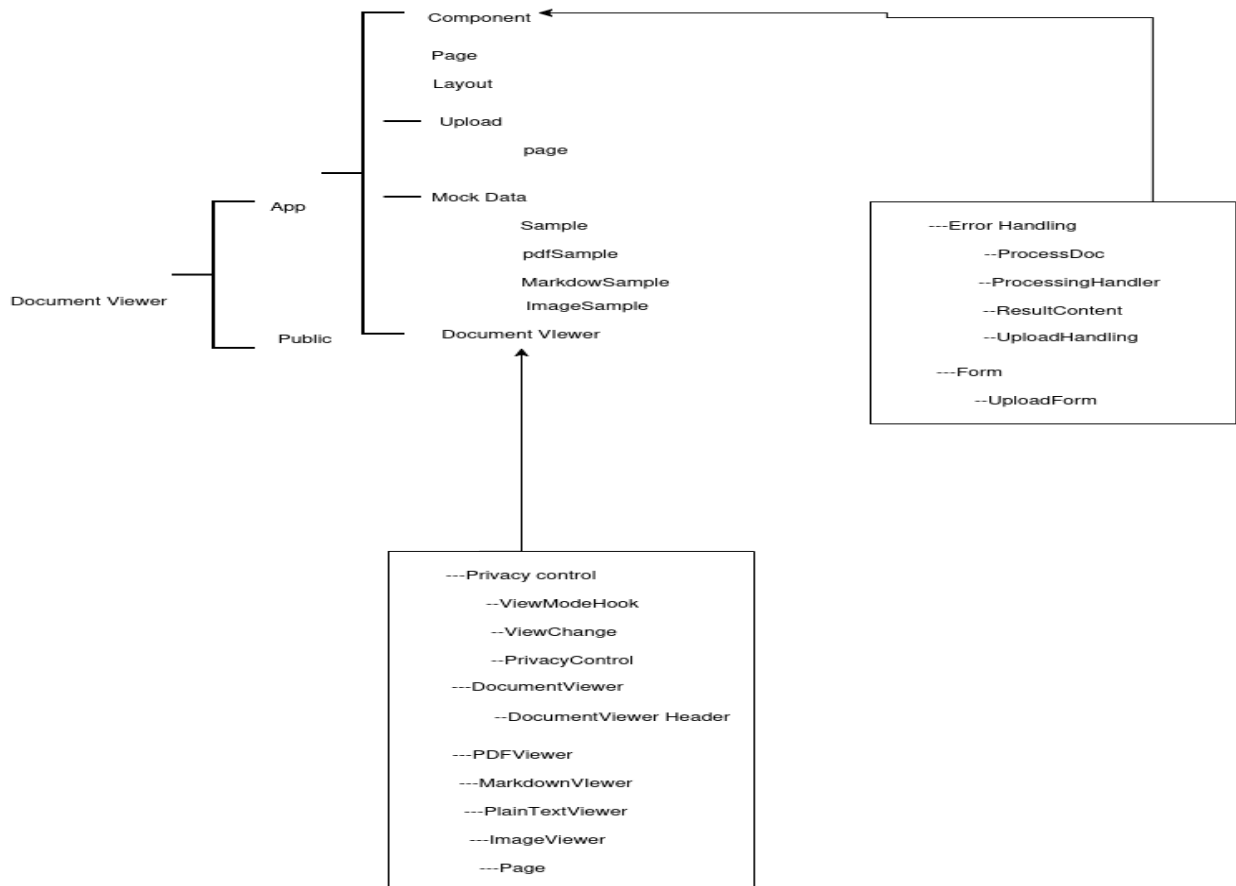
Application architecture

The frontend is built using a modular architecture where each application is an independent module. The Document Viewer redirects users to the Dashboard module when they need to access dashboard features.

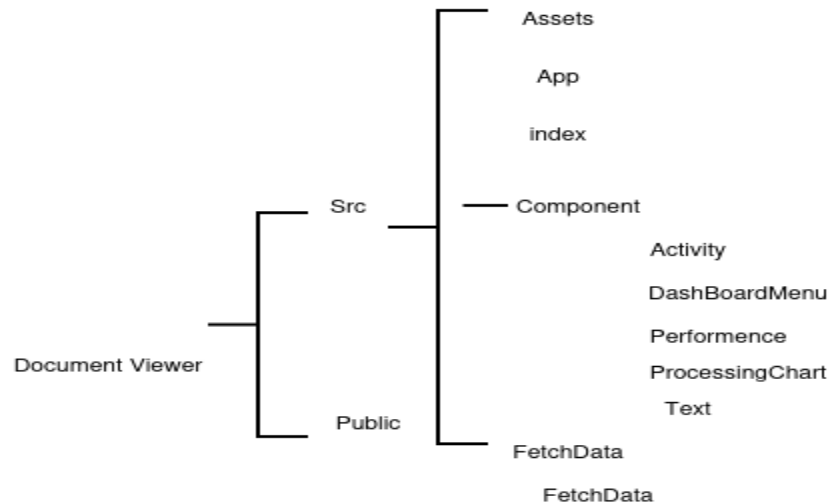
DataFlow



Document Viewer Data flow



Document Viewer Component



Dashboard Component

Technology Justification:

React and Next(With typescript): React and Next are SPA (Single Page Application) Javascript libraries and frameworks that use state to manage the UI rendering. The state management is dynamic and makes the application easy to render and access to the data. Type reduces the bug in the type verification and management in the application; it helps to track and find the bug easily. As a single-page application, React and Next are adequate for the application data flow and routing; they support a nested route, making the data transfer between pages easy.

Tailwind: Tailwind is a framework for web application styling, it's easy to use by using className name with a responsive feature. It simplifies the styling process through the use of utility class names and includes powerful, built-in responsive design capabilities.

Bundle size VS Features:

Application bundle size and feature scope are key points to consider before production deployment. While extra features bring more value to the project, they often make the bundle larger, which can affect performance, increase loading time, and require more resources for deployment. These two factors need to be carefully studied before development, depending on the needs of the application and the goals of the project.

Scalability consideration:

When scaling the application from 10,000 to 100,000 users, many things need to be considered to keep the app working well. The frontend should manage large data efficiently using methods like scrolling only what is visible and splitting long lists of pages, while caching can reduce some requests to the server. Network use should be optimized by limiting the number of files to upload and using step by step operations. The architecture also needs to support many users at once, with a strong state management and clear error handling.